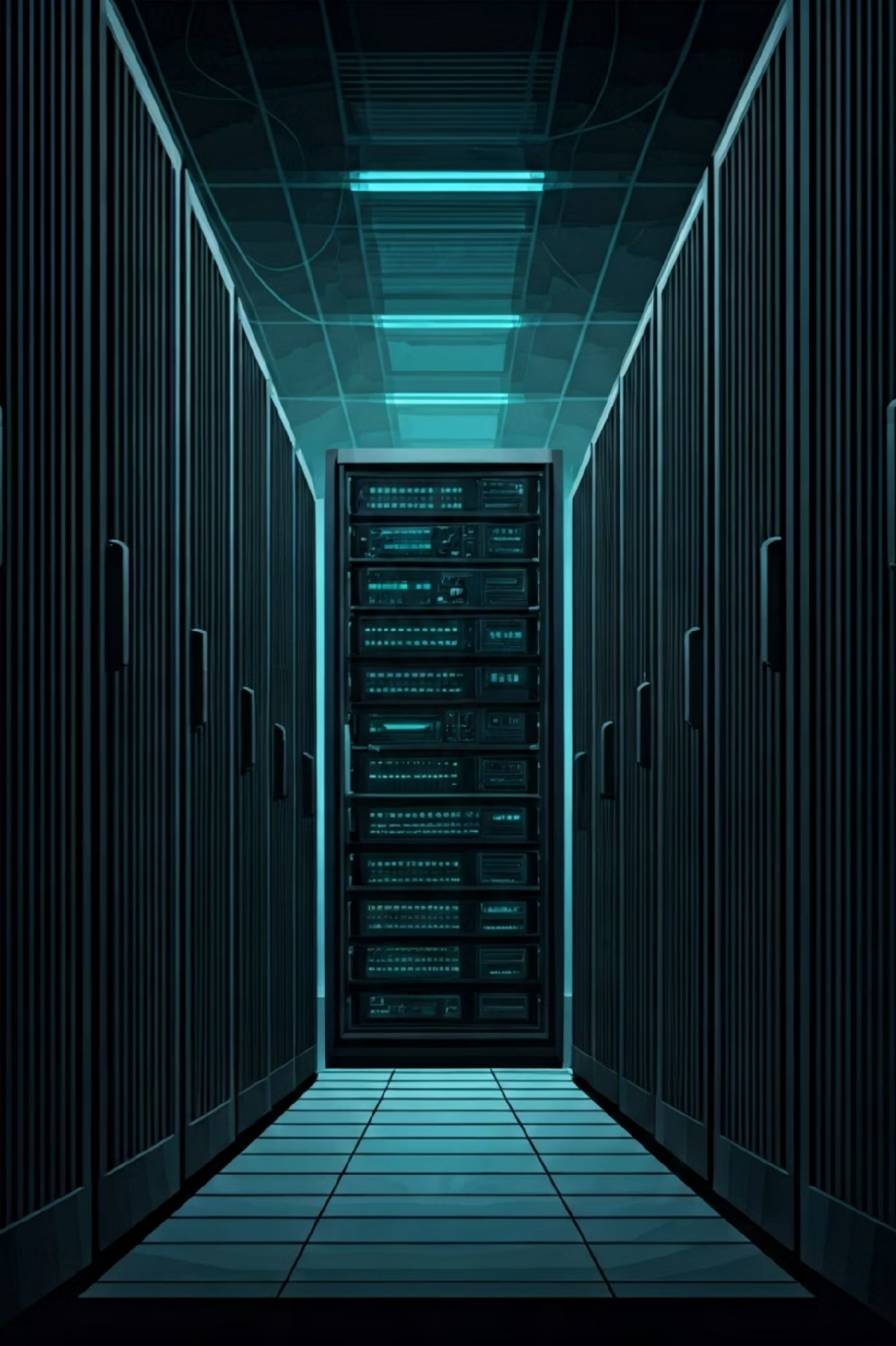




Escalabilidade Horizontal e Balanceamento de Carga

API Acadêmica com Node.js, NGINX e PostgreSQL

ARQUITETURA DE SOFTWARE — 5º PERÍODO

PROF. FILIPE TORIO LOPES RUAS NHIMI

Problema e Objetivos do Projeto



O Cenário

A universidade possui uma API acadêmica para gestão de matrículas, notas e inscrições. Em períodos críticos como matrículas online, o volume de acessos cresce dramaticamente em curto intervalo.

O Problema

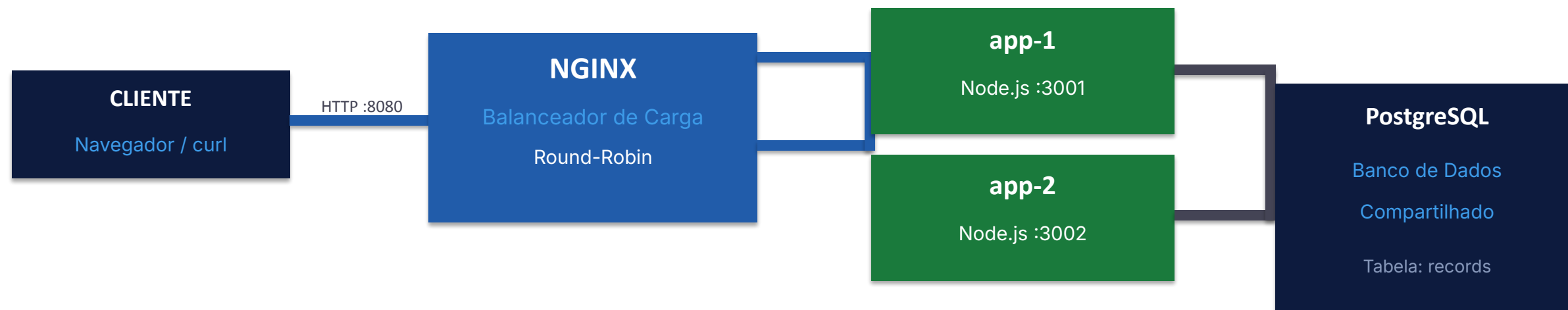
Instância única representa gargalo e ponto único de falha: queda = sistema indisponível, alta carga = lentidão para todos, manutenção = interrupção do serviço.

Objetivo

Distribuir requisições entre múltiplas instâncias idênticas usando balanceador de carga para garantir disponibilidade e performance.

Arquitetura da Solução

Visão geral dos componentes e fluxo de requisições



Node.js 20 + Express

API stateless com CRUD completo e metricas por instancia

PostgreSQL 15

Estado compartilhado — todas as instâncias acessam os mesmos dados

NGINX

Balanceador de carga com algoritmo round-robin e detecção de falhas

Docker Compose

Orquestracao: app1, app2, db e lb em containers isolados

Tecnologias e Ferramentas Utilizadas



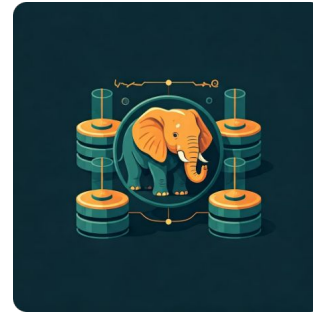
Node.js 20 + Express

Runtime JavaScript para construção da API RESTful com endpoints CRUD completos e sistema de métricas integrado



NGINX

Servidor web atuando como reverse proxy e load balancer com algoritmo round-robin e timeout de 1 segundo



PostgreSQL 15

Banco de dados relacional compartilhado entre instâncias, garantindo consistência e estado centralizado



Docker Compose

Orquestração de containers para gerenciar múltiplas instâncias da aplicação, banco de dados e balanceador

Escalabilidade: Horizontal vs Vertical

Duas abordagens para lidar com crescimento de carga

VERTICAL (Scale Up)

Aumentar recursos de UM servidor

- Mais CPU, mais RAM, SSD mais rapido
- Mesma instância da aplicação
- Sem alteração no código

Limitacoes:

- Custo cresce exponencialmente
- Existe um limite fisico de hardware
- Ponto único de falha permanece
- Manutencao exige parada total

HORIZONTAL (Scale Out) [ADOTADO]

Adicionar MAIS instancias identicas

- Mesmos recursos por servidor
- Balanceador distribui a carga
- Qualquer numero de instancias

Vantagens:

- Custo linear (paga pelo que usa)
- Sem limite pratico de crescimento
- Falha de uma não derruba o sistema
- Atualizações sem downtime (rolling)

Stateless: O Pilar da Escalabilidade Horizontal

Por que aplicações escaláveis não devem guardar estado local

STATEFUL — problemas

Estado mantido em memória local de cada instancia:

SESSOES INCONSISTENTES

Usuario loga em app-1. Proxima requisicao vai para app-2, que nao sabe que ele esta logado.

CACHE DESATUALIZADO

app-1 atualiza um registro. app-2 ainda serve o valor antigo do cache.

PERDA DE DADOS

Instancia cai, tudo em memória se perde.

STATELESS — solucao adotada

Estado externalizado no banco de dados:

- Nenhum dado de negócio em memória
- PostgreSQL e o unico repositório de verdade dos dados
- Qualquer instância pode atender qualquer requisicao
- Resultado identico independente de qual instancia respondeu

NGINX pode enviar req 1 para app-1 e req 2 para app-2; os dados sao os mesmos para ambas.

Principais Funcionalidades Implementadas - Endpoints da API

CRUD completo com identificação de instância em todas as respostas

Metodo	Rota	Descricao
GET	/records	Lista todos os registros academicos
GET	/records/:id	Consulta um registro especifico por ID
POST	/records	Cadastra novo registro (name, type, detail)
PUT	/records/:id	Atualiza um registro existente
DELETE	/records/:id	Remove um registro
GET	/health	Saúde da instancia e conexão com o banco
GET	/metrics	Contador de requisicoes e tempo medio por instancia

Identificacao de instancia: todas as respostas incluem o campo "instanceId" no JSON e o cabeçalho HTTP "x-instance-id".

NGINX — Balanceamento Round-Robin

Como o balanceador distribui as requisições entre as instâncias

nginx.conf

```
upstream api_cluster {  
    server app1:3000;  
    server app2:3000;  
}  
  
server {  
    listen 80;  
  
    location /records {  
        proxy_pass http://api_cluster;  
    }  
    location /health {  
        proxy_pass http://api_cluster;  
    }  
    proxy_connect_timeout 1s;  
}
```

Como funciona o Round-Robin

Distribuicao ciclica

Req 1 para app-1, req 2 para app-2, req 3 para app-1...

Carga equitativa

Cada instância recebe aproximadamente 50% das requisições.

Detecção de falha

Timeout de 1s. Instancia indisponivel e removida do pool.

Recuperacao automatica

Ao voltar, a instância e reincorporada sem intervenção manual.

Stateless obrigatorio

Round-robin não garante mesma instância para mesmo usuário.

Monitoramento e Metricas

Observabilidade por instancia e dashboard em tempo real

/health

Saude da instancia

- status: ok / error
- instancelid: app-1 ou app-2
- db: connected / error

Usado pelo NGINX para detectar instancias vivas.

/metrics

Metricas por instancia

- instancelid
- requestCount
- averageResponseMs

Cada instancia conta suas proprias requisicoes.

Frontend

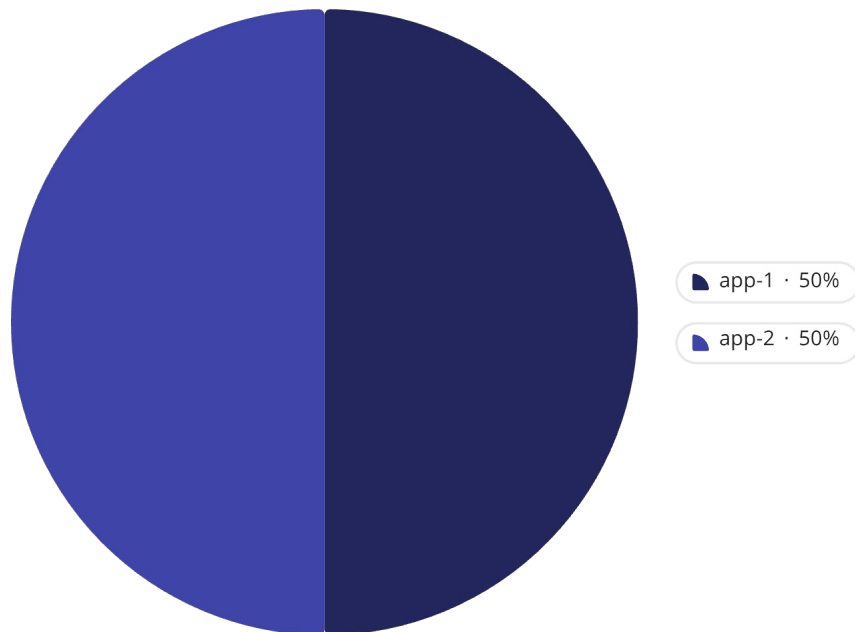
Dashboard em tempo real

- Status verde/vermelho
- Grafico de distribuicao
- Log de cada requisicao
- Teste de carga integrado

Atualiza a cada 8 segundos.

Metricas obrigatorias cobertas: multiplas instancias, distribuicao de requisicoes, identificacao de instancia, tempo medio de resposta, comportamento sob carga.

Resultados Obtidos: Distribuição de Carga



20 Requisições Simultâneas

Distribuição perfeita: 50% para cada instância.
Round-robin alterna automaticamente entre instâncias.

Extrato do Log de Requisições

```
REQ 1 -> app-1 | 15 ms  
REQ 2 -> app-2 | 12 ms  
REQ 3 -> app-1 | 14 ms  
REQ 4 -> app-2 | 13 ms  
REQ 5 -> app-1 | 16 ms  
...
```

O que isso prova

- NGINX funciona como ponto único de entrada
- Nenhuma instância fica sobrecarregada
- Campo instancelid confirma qual instância processou
- Com N instâncias, cada processa 1/N da carga

Demonstracao: Resiliência a Falhas

Comportamento do sistema quando uma instância é removida

PASSO 1 — Sistema em operação normal

Duas instancias ativas. NGINX distribui em round-robin. 50% para cada instancia.

PASSO 2 — Derrubar app-1 (docker compose stop app1)

NGINX tenta conectar em app-1. Timeout de 1 segundo. Instancia marcada como indisponivel.
100% das requisicoes passam automaticamente para app-2. Nenhuma requisicao e perda pelo usuario.

PASSO 3 — Restaurar app-1 (docker compose start app1)

app-1 volta ao ar. NGINX detecta a recuperacao e a reincorpora ao pool.
Distribuição round-robin e retomada automaticamente sem nenhuma configuração manual.

Conclusao: A escalabilidade horizontal elimina o ponto único de falha. O serviço permanece disponível mesmo com perda de instâncias — impossível em uma arquitetura de instância única.

Aprendizados da Equipe

Vantagens Confirmadas

Alta Disponibilidade

Falha de uma instância não derruba o serviço

Escalabilidade Linear

Adicionar instâncias aumenta throughput proporcionalmente

Deploys sem Downtime

Atualizar uma instância por vez (rolling update)

Custo Granular

Alocar apenas instâncias necessárias para carga atual

Limitações e Considerações

- **Banco de dados:** Com muitas instâncias, o BD pode se tornar gargalo. Solução: connection pooling, réplicas de leitura
- **Consistência eventual:** Com cache distribuído, pode haver leituras de dados ligeiramente desatualizados
- **Complexidade operacional:** Mais componentes exigem monitoramento sofisticado
- **NGINX como SPOF:** Balanceador pode ser ponto único de falha. Solução: NGINX em cluster



Arquitetura extensível: adicionar terceira instância exige apenas uma linha no docker-compose.yml e uma no nginx.conf, sem alteração no código.

Conclusao

Requisitos atendidos

CRUD completo, multiplas instancias, balanceamento, identificacao por requisicao, métricas e tolerância a falhas.

Stateless por design

Nenhum estado em memória local. PostgreSQL compartilhado garante consistencia entre instancias.

Arquitetura extensivel

Adicionar uma terceira instância exige apenas uma linha no docker-compose.yml e uma linha no nginx.conf. Sem alteração no código.

Observabilidade

Dashboard em tempo real, endpoints /health e /metrics, e identificação de instância em toda resposta HTTP.